Computer Graphics

University Campus Project

Date:
5 May 2026

Group Members

Student Name	Student ID
Shouq Mohammed Al-adwani	444007146
Aryam Fayez Almabadi	444007571
Raneem Ahmed Alghamdi	44411096
Aryam Adel Barbari	444006383

Instructor:
Dr. Ghada Alfattni

Introduction

Computer graphics is an important field in computer science that focuses on generating and manipulating visual content using programming techniques. In this project, we developed a simple simulation of a university campus using C++ graphics libraries.

The main idea of this project is to visually represent a university environment that includes building, road, sun, parking area, student, and a moving bus. The program demonstrates how basic geometric shapes such as circles, rectangles, and lines can be combined to create a complete animated scene.

This project helps in understanding the fundamentals of 2D graphics programming, including shape rendering, color filling, animation, and double buffering techniques to create smooth motion.

Methodology

The project was implemented using C++ programming language with the Graphics library (graphics.h). The main idea was to build a complete animated university campus scene by combining different geometric shapes and functions.

The program was divided into several functions to organize the code and make it easier to understand and maintain. Each function is responsible for drawing a specific part of the scene such as the sun, road, building, parking area, bus, and student.

Animation was achieved using a loop inside the main function, where the bus position is continuously updated to create a moving effect. Double buffering was also used (setactivepage and setvisualpage) to ensure smooth animation without flickering.

Basic graphics functions such as line, rectangle, circle, putpixel, and floodfill were used to construct the visual elements of the campus.

Algorithm

The following steps summarize the execution flow of the program:

1. Initialize the graphics system and set the display mode

2. Define the initial positions of dynamic elements:
- Bus position
- Animation page variables

3. Initialize control variables:
- Bus movement speed
- Page switching variable for double buffering

4. Enter the main program loop:

4.1 Clear the screen.

4.2 Draw static elements:
- Sun
- Road
- University building
- Parking area
- Student

4.3 Draw dynamic elements:
- Moving bus

4.4 Update object positions:
- Move the bus gradually from left to right across the screen.
- Reset the bus position when it exits the screen boundary.

4.5 Handle animation system:
- Switch between active page and visual page to achieve smooth animation.

4.6 Control animation speed:
- Apply a short delay using Sleep function.

5. Repeat the loop continuously to maintain animation.

6. Exit the loop when the program is terminated manually.

7. Close the graphics window and end the program execution.

Discussion

The implemented program successfully demonstrates a simple animated university campus scene using 2D computer graphics. The scene combines both static and dynamic elements to create a realistic visualization of a campus environment.

Static objects such as the university building, road, sun, parking area, and student remain fixed on the screen, while the bus is the main animated object that moves across the screen continuously.

The use of modular functions improved code organization and readability, as each part of the scene is handled separately. This makes the program easier to understand, modify, and maintain.

Double buffering (setactivepage and setvisualpage) was used to reduce flickering and provide smooth animation. Additionally, the Sleep function was used to control the speed of the animation and make the movement more natural.

Overall, the project demonstrates the basic concepts of computer graphics such as shape drawing, color filling, animation, and object movement.

Code & Demo

Code Implementation:

```c
#include <graphics.h>
#include <windows.h>

void circlePlot(int xc, int yc, int x, int y)
{
    putpixel(xc + x, yc + y, WHITE);
    putpixel(xc - x, yc + y, WHITE);
    putpixel(xc + x, yc - y, WHITE);
    putpixel(xc - x, yc - y, WHITE);
    putpixel(xc + y, yc + x, WHITE);
    putpixel(xc - y, yc + x, WHITE);
    putpixel(xc + y, yc - x, WHITE);
    putpixel(xc - y, yc - x, WHITE);
}

void midpointCircle(int xc, int yc, int r)
{
    int x = 0, y = r;
    int p = 1 - r;

    circlePlot(xc, yc, x, y);

    while (x < y)
    {
        x++;

        if (p < 0)
            p = p + 2 * x + 1;
        else
        {
            y--;
            p = p + 2 * (x - y) + 1;
        }

        circlePlot(xc, yc, x, y);
    }
}

void drawSun()
{
    midpointCircle(575, 65, 35);
    setfillstyle(SOLID_FILL, YELLOW);
    floodfill(575, 65, WHITE);
```

Figure 1:

Implementation of basic circle drawing functions used to construct circular objects such as the sun.

```
46  void drawRoad()
47  {
48      setcolor(WHITE);
49
50      rectangle(0, 310, 639, 360);
51      setfillstyle(SOLID_FILL, LIGHTGRAY);
52      floodfill(10, 320, WHITE);
53
54      rectangle(0, 360, 639, 479);
55      setfillstyle(SOLID_FILL, DARKGRAY);
56      floodfill(10, 370, WHITE);
57
58      for (int i = 20; i < 639; i += 80)
59          line(i, 420, i + 40, 420);
60  }
61
62  void drawBuilding()
63  {
64      setcolor(WHITE);
65
66      rectangle(100, 90, 470, 310);
67      setfillstyle(SOLID_FILL, LIGHTGRAY);
68      floodfill(110, 100, WHITE);
69
70      line(100, 90, 285, 40);
71      line(285, 40, 470, 90);
72      line(100, 90, 470, 90);
73
74      setfillstyle(SOLID_FILL, RED);
75      floodfill(285, 70, WHITE);
76
77      rectangle(230, 120, 340, 144);
78      setfillstyle(SOLID_FILL, WHITE);
79      floodfill(235, 125, WHITE);
80
81      setcolor(BLACK);
82      char universityText[] = "UNIVERSITY";
83      outtextxy(242, 125, universityText);
84      setcolor(WHITE);
85
86      rectangle(250, 235, 310, 310);
87      setfillstyle(SOLID_FILL, BROWN);
88      floodfill(255, 240, WHITE);
```

Figure 2:

Implementation of static graphical elements including the university building and road.

```
118 void drawParking()
119 {
120     setcolor(WHITE);
121
122     rectangle(500, 245, 635, 310);
123     setfillstyle(SOLID_FILL, BLACK);
124     floodfill(510, 250, WHITE);
125
126     line(535, 245, 535, 310);
127     line(570, 245, 570, 310);
128     line(605, 245, 605, 310);
129
130     rectangle(525, 220, 610, 245);
131     setfillstyle(SOLID_FILL, BLACK);
132     floodfill(530, 225, WHITE);
133
134     setcolor(WHITE);
135     char parkingText[] = "PARKING";
136     outtextxy(540, 227, parkingText);
137 }
138
139 void drawStudent(int x, int y)
140 {
141     setcolor(WHITE);
142
143     midpointCircle(x, y, 6);
144     line(x, y + 6, x, y + 28);
145     line(x, y + 14, x - 10, y + 23);
146     line(x, y + 14, x + 10, y + 23);
147     line(x, y + 28, x - 9, y + 42);
148     line(x, y + 28, x + 9, y + 42);
149 }
150
151 void drawBus(int x)
152 {
153     setcolor(WHITE);
154
155     rectangle(x, 375, x + 80, 415);
156     setfillstyle(SOLID_FILL, BLUE);
157     floodfill(x + 5, 380, WHITE);
158
159     rectangle(x + 10, 385, x + 28, 398);
160     rectangle(x + 35, 385, x + 53, 398);
```

Figure 3:

Implementation of additional campus components such as the parking area, student figure, and animated bus object.

```
175
176  int main()
177  {
178      int gd = DETECT, gm;
179      initgraph(&gd, &gm, (char*)"");
180
181      int busX = -80;
182      int page = 0;
183
184      while (true)
185      {
186          setactivepage(page);
187          setvisualpage(1 - page);
188
189          setbkcolor(LIGHTBLUE);
190          cleardevice();
191
192          drawSun();
193          drawRoad();
194          drawBuilding();
195          drawParking();
196          drawBus(busX);
197          drawStudent(390, 315);
198
199          busX += 5;
200
201          if (busX > 640)
202              busX = -80;
203
204          page = 1 - page;
205
206          Sleep(50);
207      }
208
209      closegraph();
210      return 0;
211  }
```

Figure 4:

Implementation of the main function controlling the overall execution of the program, including graphics initialization, double buffering technique, rendering of all campus elements, and animation of the bus movement using a continuous loop.

Demo Output:

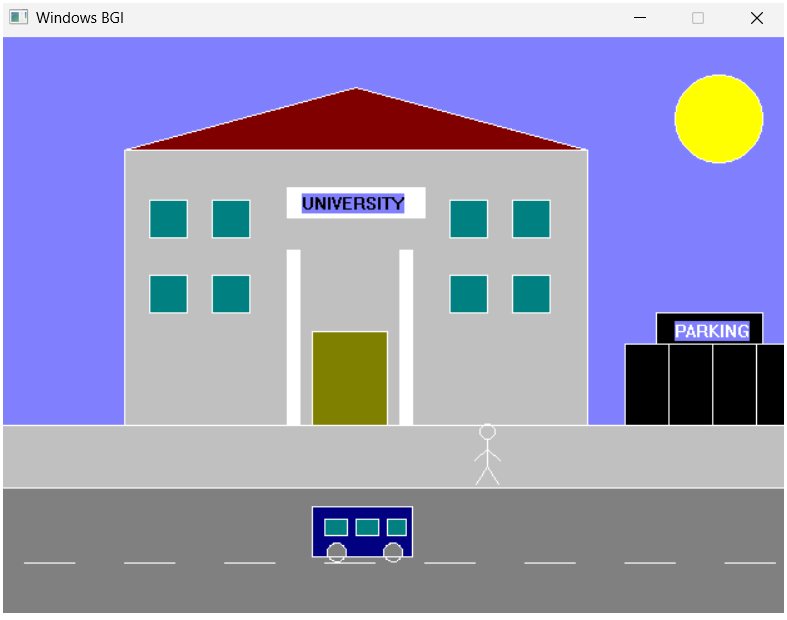


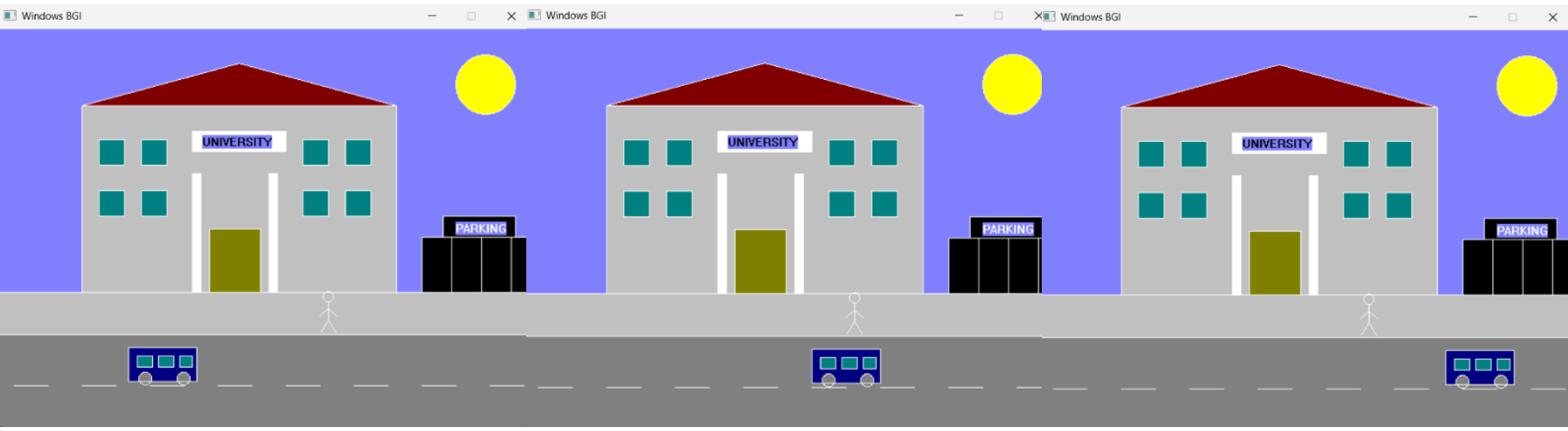
Figure 5:

Static Version

Final output of the program showing the university campus scene, including the building, road, sun, parking area, student, and bus. This figure presents the complete static layout of the designed environment.

Figure 6 :

Dynamic Version



Conclusion

In conclusion, this project successfully demonstrates the fundamentals of computer graphics by creating a simple animated university campus scene. The program effectively combines static and dynamic elements to represent a realistic environment using basic geometric shapes and graphics functions.

The use of modular programming helped in organizing the code and making it easier to understand and modify. Additionally, techniques such as double buffering improved the visual output by reducing flickering and ensuring smooth rendering.

This project enhanced our understanding of 2D graphics programming, animation concepts, and object rendering in C++.